SGLang Project Analysis: Version and Feature Timeline

Cutoff: 2026-10-19 UTC · Latest official stable: v0.5.21 (2026-10-02) · Source mode: cache

Executive assessment

vLLM is analyzed from official releases and current official documentation. Version transitions are grouped by explicit editorial boundaries; feature claims retain release-level source links.

Project positioning and current architecture

This report does not infer a current SGLang process topology from release notes alone; consult the configured official documentation for the current architecture. Current architecture sources: home

Evolution at a glance

Runtime foundation: OpenAI-compatible serving **High-performance engine**: Chunked Prefill **DeepSeek and distributed serving**: DeepSeek V3/R1 optimization **Platform and hybrid models**: HiCache **Production architectures**: Elastic EP

Detailed version and Feature timeline

Release	UTC date	Kind	Feature evidence / source
v0.1.5	2024-01-18	stable	Fix for T4 GPUs by @Ying1123 in https://github.com/sgl-project/sglang/pull/16 Fix streaming by @merrymercy in https://github.com/sgl-project/sglang/pull/30
v0.1.6	2024-01-21	stable	Add OpenAI-compatible API server (Completion and ChatCompletion) Fix sgl.select
v0.1.11	2024-02-03	stable	Faster JSON decoding [blog](https://lmsys.org/blog/2024-02-05-compressed-fsm/) Fix the error message and dependency of openai backend by @merrymercy in https://github.com/sgl-project/sglang/pull/71
v0.1.12	2024-02-11	stable	Fast JSON Decoding ([blog](https://lmsys.org/blog/2024-02-05-compressed-fsm/)) Multiple bug fixes
v0.1.13	2024-03-11	stable	correct reference dtype openai.py by @yaya-sy in https://github.com/sgl-project/sglang/pull/181 Fix server launch for jupyter notebook by @merrymercy in https://github.com/sgl-project/sglang/pull/186
v0.1.16	2024-05-14	stable	Marlin quantization kernels Many bug fixes
v0.1.17	2024-06-08	stable	Add speculative execution for OpenAI API 250 Add speculative execution for OpenAI API 250
v0.1.18	2024-07-04	stable	2x large batch prefill improvement with the new flashinfer kernels 579 2x large batch prefill improvement with the new flashinfer kernels 579
v0.1.20	2024-07-14	stable	Enable CUDA graph by default. It brings 1.5x - 2x speedup for small batch size decoding (612) Enable CUDA graph by default. It brings 1.5x - 2x speedup for small batch size decoding (612)
v0.2.0	2024-07-25	stable	We performed extensive engineering to improve the base performance. Compared to TensorRT-LLM and vLLM, SGLang now consistently delivers superior or competitive performance in both online and offline scenarios, handling models from Llama-8B to Llama-405B, on A1 New models: Llama3 405B, Deepseek MoE, InternLM, GPTBigCode, Mistral-Nemo
v0.2.5	2024-07-26	stable	We recently released a [blog](https://lmsys.org/blog/2024-07-25-sglang-llama3/). Compared to TensorRT-LLM and vLLM, SGLang Runtime consistently delivers superior or competitive performance in both online and offline scenarios, handling models from Llama-8B to We recently released a [blog](https://lmsys.org/blog/2024-07-25-sglang-llama3/). Compared to TensorRT-LLM and vLLM, SGLang Runtime consistently delivers superior or competitive performance in both online and offline scenarios, handling models from Llama-8B to
v0.2.9	2024-08-02	stable	New feature: Chunked prefill (800, 811) New feature: Chunked prefill (800, 811)
v0.2.13	2024-09-19	stable	New Feature: Support window attention for Gemma-2 (1056 1090 1112), enable chunked-prefill by default (1040 984), support all sampling penalties (973) New Feature: Support window attention for Gemma-2 (1056 1090 1112), enable chunked-prefill by default (1040 984), support all sampling penalties (973)
v0.3.0	2024-09-19	stable	Up to 7x higher throughput for DeepSeek Multi-head Latent Attention (MLA) Up to 1.5x lower latency with torch.compile on small batch sizes
v0.3.2	2024-10-02	stable	Support torch.compile, cuda graph for triton attention backend and DeepSeek MLA 1442 1422 Support torch.compile, cuda graph for triton attention backend and DeepSeek MLA 1442 1422
v0.3.4.post1	2024-10-22	post	Hosted the first LMSYS online meetup: Efficient LLM Deployment and Serving. Covered CPU overhead hiding, faster constrained decoding, and DeepSeek MLA. [Slides](https://github.com/sgl-project/sgl-learning-materials?tab=readme-ov-filethe-first-lmsys-online-meetup-efficient-llm-deployment-and-serving)
v0.3.6	2024-11-22	stable	Reduce CPU overhead by enabling overlap scheduler by default. 1.1x higher throughput. (2105, 2067, 2095) Reduce CPU overhead by enabling overlap scheduler by default. 1.1x higher throughput. (2105, 2067, 2095)
v0.4.0	2024-12-04	stable	Zero-overhead batch scheduler: 1.1x increase in throughput. Data parallelism attention for DeepSeek models: up to 1.9x decoding throughput improvement.

Release	UTC date	Kind	Feature evidence / source
v0.4.1	2024-12-25	stable	We're excited to announce SGLang v0.4.1, which now supports [DeepSeek V3](https://huggingface.co/deepseek-ai/DeepSeek-V3) - currently the strongest open-source LLM, even surpassing GPT-4o. The SGLang and DeepSeek teams worked together to get DeepSeek V3 FP8 running on NVIDIA and AMD GPU from day one. We've also supported MLA optimization and DP attention before, making SGLang one of the best open-source LLM engines for running DeepSeek models.
v0.4.3	2025-02-14	stable	The SGLang team is excited to announce the release of v0.4.3. We will keep improving DeepSeek V3/R1 performance. In the last six weeks, SGLang has been the fastest engine running DeepSeek V3/R1 among all open-source LLM inference engines. We stay ahead by inte The SGLang team is excited to announce the release of v0.4.3. We will keep improving DeepSeek V3/R1 performance. In the last six weeks, SGLang has been the fastest engine running DeepSeek V3/R1 among all open-source LLM inference engines. We stay ahead by inte
v0.4.4	2025-03-13	stable	The SGLang team is excited to announce the release of v0.4.4. We will keep improving DeepSeek V3/R1 performance. With the combination of FlashInfer, MTP, DeepGEMM, and Torch Compile optimizations on H200, it can achieve nearly 100 tokens/s, which is currently The SGLang team is excited to announce the release of v0.4.4. We will keep improving DeepSeek V3/R1 performance. With the combination of FlashInfer, MTP, DeepGEMM, and Torch Compile optimizations on H200, it can achieve nearly 100 tokens/s, which is currently
v0.4.5	2025-04-07	stable	The SGLang team is excited to the release of v0.4.5! This version introduces several significant features, including Llama 4 support, FlashAttention 3 backend, EAGLE3 speculative decoding, DeepEP integration, and disaggregated prefill and decoding. The SGLang team is excited to the release of v0.4.5! This version introduces several significant features, including Llama 4 support, FlashAttention 3 backend, EAGLE3 speculative decoding, DeepEP integration, and disaggregated prefill and decoding.
v0.4.6	2025-04-27	stable	Use FlashAttention3 as the default attention backend for main stream models (DeepSeek, Qwen, Llama, etc). https://github.com/sgl-project/sglang/issues/4709issuecomment-2817728855 Use FlashAttention3 as the default attention backend for main stream models (DeepSeek, Qwen, Llama, etc). https://github.com/sgl-project/sglang/issues/4709issuecomment-2817728855
v0.4.7	2025-06-11	stable	The PD disaggregation and large-scale EP functionalities from the [blog post](https://lmsys.org/blog/2025-05-05-large-scale-ep/) have now been fully merged into the latest release. PD disaggregation and large-scale EP, in addition to supporting DeepSeek V3/R1, now also support Qwen 3 in the latest release.
v0.4.8	2025-06-24	stable	OpenAI-Compatible Server Refactor Re-structured the OpenAI-compatible server to support production and enterprise environments. Key improvements include:
v0.4.10	2025-07-31	stable	SpecForge: Accelerating Speculative Decoding Training for SGLang https://lmsys.org/blog/2025-07-25-spec-forge/ Deploying Kimi K2 with PD Disaggregation and Large-Scale Expert Parallelism on 128 H200 GPUs
v0.5.1	2025-08-23	stable	[bugifx] QWen-1M context support[2/3] using current cuda stream in the DCA's kernel for bugfix. by @sighingnow in https://github.com/sgl-project/sglang/pull/8611 [bugifx] QWen-1M context support[2/3] using current cuda stream in the DCA's kernel for bugfix. by @sighingnow in https://github.com/sgl-project/sglang/pull/8611
v0.5.2	2025-09-12	stable	SGLang HiCache: Fast Hierarchical KV Caching with Your Favorite Storage Backends: https://lmsys.org/blog/2025-09-10-sglang-hicache/ fix(grok): remove duplicate replicatelmhead configuration by @vincentzed in https://github.com/sgl-project/sglang/pull/9549
v0.5.3	2025-10-06	stable	Day 0 Support for DeepSeek-V3.2 with Sparse Attention: https://lmsys.org/blog/2025-09-29-deepseek-V32/ Integration of FlashAttention 4 prefill kernels
v0.5.4	2025-10-26	stable	Model gateway v0.2 release: https://docs.sglang.ai/advancedfeatures/router.html Model gateway v0.2 release: https://docs.sglang.ai/advancedfeatures/router.html
v0.5.5	2025-11-06	stable	Video and image generation support https://lmsys.org/blog/2025-11-07-sglang-diffusion/ Blackwell kernel optimizations and MoE runner backend refactor
v0.5.6	2025-12-03	stable	Support for DeepSeek V3.2/V3.2 Speciale 14249 Blockwise diffusion language model support 12588

Release	UTC date	Kind	Feature evidence / source
v0.5.7	2026-01-01	stable	Day 0 Support for LLaDA 2.0: https://lmsys.org/blog/2025-12-19-diffusion-llm/ [SGLang-Diffusion] Day 0 Support for Qwen-Image-Edit-2509, Qwen-Image-Edit-2511, Qwen-Image-2512 and Qwen-Image-Layered
v0.5.8	2026-01-23	stable	Up to 1.5x faster across the board for all major diffusion models https://lmsys.org/blog/2026-01-16-sglang-diffusion/ Optimizing GLM4-MoE for Production: 65% Faster TTFT https://lmsys.org/blog/2026-01-21-novita-glm4/
v0.5.9	2026-02-24	stable	LoRA Weight Loading Overlap with Computation: Overlap LoRA weight loading with computation during inference, reducing TTFT by ~78% and TPOT by ~34.88% on large adaptors: 15512 TRT-LLM NSA Kernel Integration for DeepSeek V3.2: Integrate TRT-LLM DSA kernels for Native Sparse Attention, boosting DeepSeek V3.2 performance by 3x-5x on Blackwell platforms with trtllm for both --nsa-prefill-backend and --nsa-decode-backend
v0.5.10rc0	2026-03-28	rc	Piecewise CUDA Graph Enabled by Default: Piecewise CUDA graph capture is now the default execution mode, reducing memory overhead and improving throughput for models with complex control flow patterns: 16331 Piecewise CUDA Graph Enabled by Default: Piecewise CUDA graph capture is now the default execution mode, reducing memory overhead and improving throughput for models with complex control flow patterns: 16331
v0.5.10	2026-04-06	stable	Piecewise CUDA Graph Enabled by Default: Piecewise CUDA graph capture is now the default execution mode, reducing memory overhead and improving throughput for models with complex control flow patterns: 16331 Piecewise CUDA Graph Enabled by Default: Piecewise CUDA graph capture is now the default execution mode, reducing memory overhead and improving throughput for models with complex control flow patterns: 16331
v0.5.10.post1	2026-04-09	post	Bumps flashinfer from v0.6.7.post2 to v0.6.7.post3 to resolve an issue in its jit cubin downloader.
v0.5.11	2026-05-05	stable	CUDA 13 + Torch 2.11: Default CUDA version moves to 13.0 across SGLang, sgl-kernel, and Docker images, and PyTorch is upgraded from 2.9 to 2.11 — modernizing the build matrix and unlocking newer kernels: 21247, 24162, 24183, 23593 ([tracking issue 21498](https CUDA 13 + Torch 2.11: Default CUDA version moves to 13.0 across SGLang, sgl-kernel, and Docker images, and PyTorch is upgraded from 2.9 to 2.11 — modernizing the build matrix and unlocking newer kernels: 21247, 24162, 24183, 23593 ([tracking issue 21498](https
v0.5.12	2026-05-16	stable	DeepSeek V4 support: Full inference path for DeepSeek-V4 (23882), including: Parallelism: Tensor Parallelism/Expert Parallelism/Context Parallelism/Data Parallel Attention
v0.5.12.post1	2026-05-26	post	v0.5.12.post1 is a stability patch on top of v0.5.12. It cherry-picks 12 fixes — primarily for DeepSeek V4 — onto the release branch. DSV4-Pro emits garbled text during single-token decode on B200/B300 (fix deepgemm UE8M0 scale-packing path by ceiling activation scales before packing): 25733
v0.5.13	2026-06-13	stable	Diffusion: [Cosmos3](https://docs.sglang.io/cookbook/diffusion/Cosmos/Cosmos3), [LingBot-World](https://docs.sglang.io/cookbook/diffusion/LingBot-World/LingBot-World), [SANA-WM](https://docs.sglang.io/cookbook/diffusion/SANA-WM/SANA-WM), [Ernie-Image](https:// Spec V2 is now the default speculative-decoding path: Tree drafting with topk > 1 is production-ready across the triton / FA3 / MLA / aiter backends, including pagesize > 1 and Mamba/hybrid-linear models ([26997](https://github.com/sgl-project/sglang/pull/2699
v0.5.14	2026-06-26	stable	DeepSeek-V4 on GB300 since Day 0: 5x higher throughput at the same interactivity, serving DeepSeek-V4 on NVIDIA GB300 with SGLang ([blog](https://pytorch.org/blog/serving-deepseek-v4-on-gb300-with-sglang-5x-higher-throughput-at-the-same-interactivity-since-day Waterfill & LPLB MoE load balancing: Two dispatch-time load-balancing methods for DeepEP expert parallelism: Waterfill for shared-expert dispatch and LPLB for redundant expert replicas, improving throughput for DeepSeek-V3/R1 and DeepSeek-V4 ([blog](https://ww
v0.5.15	2026-07-10	stable	GLM-5.2 NVFP4, tuned for production: We took time this cycle to tune GLM-5.2 NVFP4 on Blackwell for optimized production serving. It now runs at 500+ tok/s/user on 8x B300, 450 on 4x GB300 (bs=1). Run GLM-5.2 with our [cookbook](https://docs.sglang.io/cookbook GLM-5.2 NVFP4, tuned for production: We took time this cycle to tune GLM-5.2 NVFP4 on Blackwell for optimized production serving. It now runs at 500+ tok/s/user on 8x B300, 450 on 4x GB300 (bs=1). Run GLM-5.2 with our [cookbook](https://docs.sglang.io/cookbook
v0.5.15.post1	2026-07-14	post	30454 30627: Fix DSA model launching on non Cuda/HIP devices 30454 30627: Fix DSA model launching on non Cuda/HIP devices

Release	UTC date	Kind	Feature evidence / source
v0.5.16	2026-07-25	stable	DSpark: confidence-driven speculative decoding: A new speculative algorithm. It drafts semi-autoregressively in blocks, then sizes each verify window from the draft's own confidence instead of a fixed draft length. Reaches 383.7 tok/s at accept length ~5 on De DSpark: confidence-driven speculative decoding: A new speculative algorithm. It drafts semi-autoregressively in blocks, then sizes each verify window from the draft's own confidence instead of a fixed draft length. Reaches 383.7 tok/s at accept length ~5 on De
v0.5.17	2026-08-08	stable	Kimi K3 day-0 support: A 2.8T-parameter multimodal LatentMoE (896 experts, top-16, routed in a 3584-dim latent space) with a 1M-token context, 69 KDA linear-attention layers interleaved with 24 MLA layers, and a MoonViT3d vision tower, shipping as a native MXF Kimi K3 day-0 support: A 2.8T-parameter multimodal LatentMoE (896 experts, top-16, routed in a 3584-dim latent space) with a 1M-token context, 69 KDA linear-attention layers interleaved with 24 MLA layers, and a MoonViT3d vision tower, shipping as a native MXF
v0.5.18	2026-08-22	stable	| Muse Glimmer | Autoregressive (Multimodal) | [34262](https://github.com/sgl-project/sglang/pull/34262) | [link](https://docs.sglang.io/cookbook/autoregressive/Meta/MuseGlimmer) | | SANA-Video | Diffusion | [32921](https://github.com/sgl-project/sglang/pull/32921) | [link](https://docs.sglang.io/cookbook/diffusion/SANA-Video/SANA-Video) |
v0.5.19	2026-09-05	stable	| LongCat-Image-Edit & Edit-Turbo | Diffusion | [35829](https://github.com/sgl-project/sglang/pull/35829) | | GLM-5.3 deployment guide: [link](https://docs.sglang.io/cookbook/autoregressive/GLM/GLM-5.3)
v0.5.20	2026-09-18	stable	| SenseNova-U1.5-8B-MoT | Diffusion | [36606](https://github.com/sgl-project/sglang/pull/36606) | [link](https://docs.sglang.io/cookbook/diffusion/SenseNova/SenseNova-U1.5-8B-MoT) | | FastH3 (4-step MiniMax-H3 distill) | Diffusion | [37480](https://github.com/sgl-project/sglang/pull/37480) | [link](https://docs.sglang.io/cookbook/diffusion/MiniMax/MiniMax-H3) |
v0.5.21	2026-10-02	stable	| DeepSeek-V4.1 Flash | LLM / VLM | [link](https://docs.sglang.io/cookbook/autoregressive/DeepSeek/DeepSeek-V41) | | DiffusionGemma | Diffusion | [link](https://docs.sglang.io/cookbook/autoregressive/Google/DiffusionGemma) |

Feature evolution by technical dimension

Dimension	Release evidence
api_protocol	v0.5.13, v0.5.14, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
correctness_stability	v0.5.15, v0.5.15.post1, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
engine_model_runner	v0.5.14, v0.5.15, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
execution_kernels	v0.5.15, v0.5.15.post1, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
hardware_platform	v0.5.15, v0.5.15.post1, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
kv_cache_memory	v0.5.14, v0.5.15, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
model_modalities	v0.5.15, v0.5.15.post1, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
observability_operations	v0.5.14, v0.5.15, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
parallelism_distributed	v0.5.15, v0.5.15.post1, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
quantization	v0.5.15, v0.5.15.post1, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
scheduling_batching	v0.5.14, v0.5.15, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21
speculative_decoding	v0.5.14, v0.5.15, v0.5.16, v0.5.17, v0.5.18, v0.5.19, v0.5.20, v0.5.21

Engineering strengths and risks

Strengths and risks must be validated against the deployment workload. In particular, removal/deprecation statements refer to the named implementation path, not necessarily to the underlying concept. The v0.25 legacy PagedAttention implementation removal does not mean block-oriented KV management ceased to exist.

Best-fit scenarios and operational cautions

Validate models, kernels, parallel topology, cache configuration, and API compatibility against the target hardware before upgrading. Pin images and dependencies for production rollouts.

Sources and methodology

Official releases: GitHub Releases. Drafts are excluded; RCs are retained in normalized data but excluded from primary stages. Post/patch releases are retained. Data cutoff: 2026-10-19.